

CAP-STN: Continuous Adaptive Spatial Propagation for Large-Scale Spatio-Temporal Forecasting

Mohammadreza Akbari Lor*, Shu-Ching Chen[†], Mei-Ling Shyu*

*School of Science and Engineering, University of Missouri-Kansas City, Kansas City, MO 64110, USA

[†]Data Science and Analytics Innovation Center (dSAIC), University of Missouri-Kansas City, Kansas City, MO 64110, USA
Email: {ma7fy, s.chen, shyum}@umkc.edu

Abstract—Spatio-temporal forecasting in large-scale systems requires modeling complex dependencies across space and time, including continuous spatial interactions and structured temporal patterns. Existing approaches based on graph neural networks and attention mechanisms often rely on discrete graph structures, limited temporal representations, and shallow spatial aggregation, which restrict their ability to capture fine-grained dynamics. This paper proposes CAP-STN (Continuous Adaptive Propagation Spatio-Temporal Network), a spatio-temporal forecasting framework that integrates periodic temporal encoding, continuous 2D spatial encoding, gated multi-hop spatial propagation, and node-adaptive fusion. The periodic encoding explicitly captures recurring temporal patterns, while the spatial encoding models translation-aware relationships directly from geographic coordinates. The proposed propagation mechanism enables controlled multi-hop aggregation within a single layer, and the adaptive fusion dynamically balances spatial and temporal contributions at the node level. CAP-STN is evaluated on the LargeST dataset and is compared with strong baselines, including MTGNN and GMAN. Experimental results demonstrate consistent improvements across multiple evaluation metrics, and ablation studies validate the contribution of each component. These results highlight the effectiveness of combining continuous spatial representations, explicit temporal modeling, and adaptive information flow for large-scale spatio-temporal forecasting.

Index Terms—Spatio-temporal Forecasting, Temporal Encoding, Spatial Encoding, Gated Multi-hop Spatial Propagation

I. INTRODUCTION

Spatio-temporal forecasting is a fundamental problem in applications such as traffic prediction, environmental monitoring, and urban analytics, where data exhibit complex dependencies across both space and time. In traffic systems, for instance, sensor measurements reflect both structured temporal patterns (e.g., daily and weekly cycles) and spatial correlations through the road network.

Recent advances in deep learning have significantly improved spatio-temporal prediction [1]. Graph neural networks (GNNs) are widely used to model spatial dependencies, while temporal dynamics are captured using recurrent, convolutional, or attention-based architectures. Hybrid models such as MTGNN [2] and GMAN [3] combine graph learning with attention mechanisms to achieve strong performance. However, several limitations remain. Existing methods rely on discrete graph structures, limiting their ability to model continuous spatial relationships. Temporal modeling often lacks

explicit representation of periodic patterns. Spatial aggregation is typically restricted to single-hop neighborhoods per layer, requiring deep stacking that may lead to over-smoothing. In addition, fusion of spatial and temporal information is usually fixed and not adaptive to node-specific dynamics.

To address these limitations, this paper proposes CAP-STN (Continuous Adaptive Propagation Spatio-Temporal Network), a spatio-temporal forecasting framework that integrates four key components: (1) periodic temporal encoding to capture recurring patterns, (2) continuous 2D spatial encoding based on a learnable Mercer expansion for translation-aware spatial representation, (3) gated multi-hop spatial propagation for controlled multi-hop aggregation within a single layer, and (4) node-adaptive fusion to dynamically balance spatial and temporal contributions.

CAP-STN is evaluated on the LargeST dataset [4], a large-scale traffic forecasting benchmark, and is compared with strong baselines, namely MTGNN and GMAN. The experimental results show consistent improvements across multiple metrics, and ablation studies validate the effectiveness of each component.

CAP-STN has the following main contributions:

- 1) Continuous 2D spatial encoding for multi-scale and translation-aware spatial relationships;
- 2) Gated multi-hop spatial propagation for efficient and stable multi-hop aggregation;
- 3) Node-adaptive fusion for dynamic balancing of spatial and temporal dependencies;
- 4) Comprehensive evaluation on a large-scale benchmark that demonstrates improved performance.

The rest of this paper is organized as follows. Section II reviews related work. The proposed framework is introduced in Section III, and the results and discussion are presented in Section IV. Finally, Section V summarizes this paper and discusses future work.

II. RELATED WORK

Spatio-temporal learning has been extensively studied, using machine learning and deep learning models [5]–[8]. Specifically for graph-based deep learning, early approaches such as STGCN [9] and T-GCN [10] combine graph convolution with

temporal modeling to capture spatial and temporal dependencies. More recent models incorporate attention mechanisms to improve expressiveness. GMAN [3] employs multi-head attention to model spatial and temporal interactions, while A3T-GCN [11] integrates attention into graph convolutional frameworks. Despite their effectiveness, these methods rely on discrete graph structures, limiting their ability to capture continuous spatial relationships.

Temporal modeling has evolved from recurrent and convolutional architectures to attention-based approaches. While positional encoding preserves sequence order, it does not explicitly capture periodic patterns common in real-world data. Recent work, such as SST-GNN [12], highlights the importance of modeling periodic temporal dynamics. In contrast, the approach explicitly incorporates periodic temporal encoding alongside positional encoding to better represent recurring patterns.

Spatial representation in most existing methods is based on adjacency matrices or learned graph structures. However, such discrete representations may fail to capture fine-grained spatial interactions. TGAT [13] introduces functional encoding to model interactions as functions of time differences, enabling continuous representations. Inspired by this idea, this work extends functional encoding to the spatial domain using a Mercer-based formulation [14], allowing the model to capture translation-aware spatial relationships.

Capturing long-range dependencies in graphs typically requires stacking multiple layers to increase the receptive field of the model, which can lead to over-smoothing. GeniePath [15] addresses this by introducing adaptive depth modeling with gated mechanisms; while multi-scale approaches like MD-GCN [16] attempt to capture hierarchical dependencies. The proposed model instead performs explicit multi-hop propagation within a single layer, enabling controlled multi-hop aggregation without increasing network depth. Finally, combining spatial and temporal information remains a key challenge. Many existing models use fixed or implicit fusion strategies [3], [9], [10], which may not adapt well to node-specific dynamics. In this work, this limitation is addressed through a node-adaptive fusion mechanism that dynamically balances spatial and temporal contributions for each node.

III. METHODOLOGY

CAP-STN integrates four key components designed to address limitations in existing spatio-temporal models, namely (1) periodic temporal encoding for structured temporal patterns, (2) continuous 2D spatial encoding for irregular spatial relationships, (3) multi-hop gated spatial propagation, and (4) node-adaptive spatio-temporal fusion. These components are designed to jointly model spatial geometry, temporal periodicity, and adaptive information flow (as shown in Figure 1).

A. Periodic Temporal Encoding

Standard positional encodings capture ordering but fail to explicitly represent periodic temporal patterns common in real-world data (e.g., daily or weekly cycles). To address this,

alongside positional encoding, explicit periodic encodings are defined as follows.

$$[\sin(tod), \cos(tod), \sin(dow), \cos(dow)] \quad (1)$$

where *tod* denotes normalized time-of-day and *dow* denotes normalized day-of-week. The normalization maps each variable into $[0, 1]$ before applying the sinusoidal transformation. The goal is to ensure that temporally distant but semantically similar time points (e.g., 9 AM on different days) are mapped close in feature space. Unlike purely positional encodings, this explicitly injects real-world periodic structure into the model.

B. Continuous 2D Spatial Encoding

Spatial relationships in real-world systems are continuous and irregular, and cannot be fully captured by discrete graph connectivity alone. Inspired by prior work on functional encoding for irregular temporal intervals [13], [14], where kernels are defined to approximate functions of time differences, this idea is extended to the spatial domain.

In particular, the goal is to construct a feature mapping such that the similarity between two locations depends on their relative displacement.

$$\kappa(\mathbf{x}, \mathbf{y}) = \psi(\mathbf{x} - \mathbf{y}) \quad (2)$$

where $x, y \in \mathbb{R}^2$ represent spatial coordinates and $\psi(\cdot)$ is a translation-invariant function.

To approximate such a kernel, a truncated Mercer expansion is used. For a one-dimensional coordinate x , the feature map is defined as:

$$\phi(x) = \left\|_{i=1}^k \left[\sqrt{c_0^{(i)}} \right. \right. \\ \left. \left. \left\{ \sqrt{c_{j,\cos}^{(i)}} \cos\left(\frac{j\pi x}{\omega_i}\right) \right\}_{j=1}^{d_h}, \right. \right. \\ \left. \left. \left\{ \sqrt{c_{j,\sin}^{(i)}} \sin\left(\frac{j\pi x}{\omega_i}\right) \right\}_{j=1}^{d_h} \right] \right. \quad (3)$$

where

- k is the number of frequency bands,
- d_h is the number of harmonics per frequency,
- $\omega_i > 0$ are learnable frequency parameters,
- $c_0^{(i)}, c_{j,\cos}^{(i)}, c_{j,\sin}^{(i)} \geq 0$ are learnable coefficients,
- $\|$ denotes concatenation across frequency bands.

The resulting feature dimension for a single coordinate is $\dim(\phi(x)) = k \times (1 + 2d_h)$. For two-dimensional coordinates $\mathbf{x} = (lon, lat) \in \mathbb{R}^2$, the encoding is applied independently and concatenated:

$$\phi_{2D}(lon, lat) = [\phi(lon), \phi(lat)] \quad (4)$$

resulting in $\dim(\phi_{2D}) = 2 \times \dim(\phi(x))$.

As shown in [14], the inner product in the feature space approximates a translation-invariant kernel:

$$\kappa(x, y) = \langle \phi(x), \phi(y) \rangle \approx \psi(x - y) \quad (5)$$

and for two-dimensional inputs:

$$\kappa(\mathbf{x}, \mathbf{y}) = \langle \phi_{2D}(\mathbf{x}), \phi_{2D}(\mathbf{y}) \rangle \approx \psi(\mathbf{x} - \mathbf{y}) \quad (6)$$

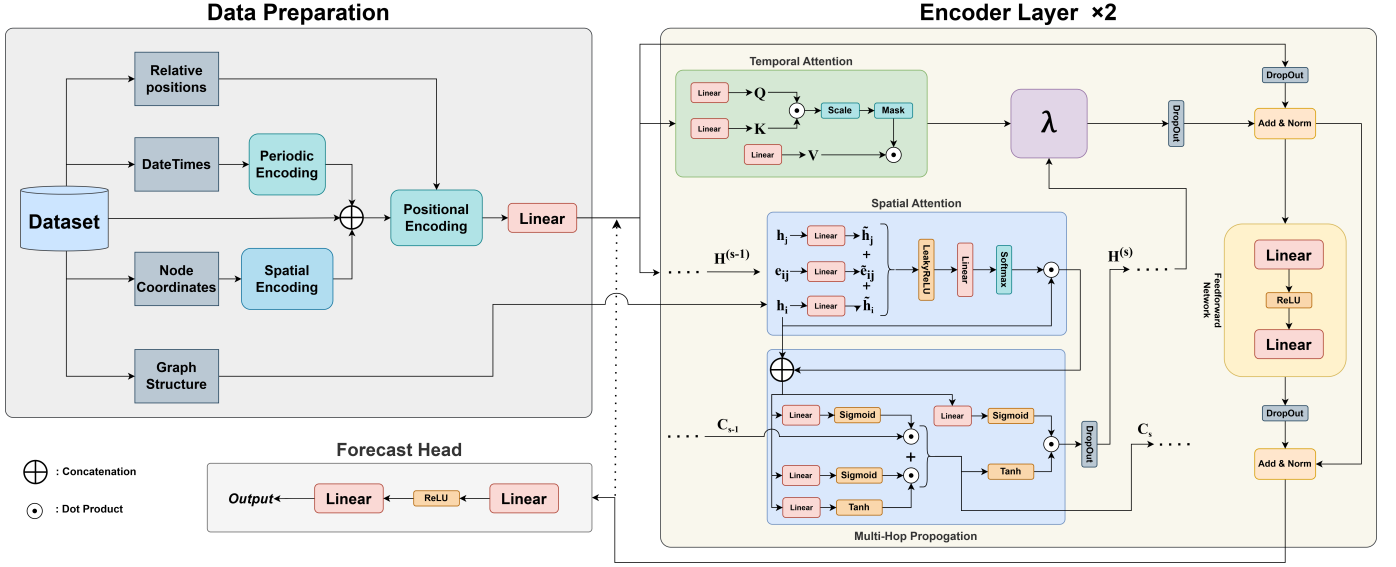


Fig. 1. Overview of the CAP-STN model architecture. The pipeline integrates periodic temporal encoding, continuous 2D spatial encoding, spatio-temporal attention, gated multi-hop spatial propagation, and node-adaptive fusion for final prediction.

Thus, the similarity between two nodes depends explicitly on their relative spatial displacement rather than on their absolute positions.

This formulation extends the idea of functional temporal encoding to 2D surfaces and, therefore, enables continuous representation of spatial relationships, multi-scale modeling through learnable frequency bands, and adaptation to irregular node distributions. This allows the model to go beyond the general graph-based approaches that rely solely on adjacency structure and instead capture fine-grained spatial interactions directly from geographic coordinates.

C. Spatio-Temporal Attention

Dependencies across time and space are modeled using a hybrid attention mechanism that combines Transformer-style temporal attention [17] with graph attention inspired by GATv2 [18].

First, a linear transform projects the data into the model's embedding space. Let the input representation be $h \in \mathbb{R}^{T \times N \times D}$, where T is the number of time steps, N is the number of nodes, and D is the embedding dimension. For a fixed node i , the associated embedding matrix $h_i \in \mathbb{R}^{T \times D}$, is obtained by slicing the tensor h along the node dimension and contains all timesteps of the input. Similarly, for a fixed timestep t , the associated embedding matrix $h^{(t)} \in \mathbb{R}^{N \times D}$, is obtained by slicing the tensor h along the time dimension and contains all nodes of the input.

1) *Temporal Attention*: Temporal dependencies are modeled independently for each node using multi-head self-attention along the time dimension. For a fixed node i , the learnable projections for each head are computed as follows.

$$Q_i^{(r)} = h_i W_q^{(r)}, \quad K_i^{(r)} = h_i W_k^{(r)}, \quad V_i^{(r)} = h_i W_v^{(r)} \quad (7)$$

where:

- $W_q^{(r)}, W_k^{(r)}, W_v^{(r)} \in \mathbb{R}^{D \times d_k}$ are trainable projection matrices,
- $r = 1, \dots, R$ and R is the number of attention heads,
- $d_k = D/R$ is the dimensionality per attention head.

The attention for each head is computed as:

$$\text{Attn}(Q_i^{(r)}, K_i^{(r)}, V_i^{(r)}) = \text{softmax} \left(\frac{Q_i^{(r)} K_i^{(r)\top}}{\sqrt{d_k}} + M \right) V_i^{(r)} \quad (8)$$

where $M \in \mathbb{R}^{T \times T}$ is a causal mask defined as:

$$M_{t,t'} = \begin{cases} 0, & t' \leq t \\ -\infty, & t' > t \end{cases} \quad (9)$$

This mask ensures that each time step attends only to past and current observations, enforcing the forward-only information flow. Equation (10) computes the multi-head attention. This produces a temporal representation for each node.

$$H_i^{\text{temporal}} = \parallel_{r=1}^R \text{Attn}_r(Q_i^{(r)}, K_i^{(r)}, V_i^{(r)}) \in \mathbb{R}^{T \times D} \quad (10)$$

2) *Spatial Attention*: Spatial dependencies are modeled independently for each time step using multi-head attention through the graph structure. For a fixed time step t , let $h^{(t)} \in \mathbb{R}^{N \times D}$ denote the node features, where N is the number of nodes and D is the embedding dimension. Let R and $d_k = D/R$ denote the number of attention heads and the feature dimension of each head, similar to the temporal attention definitions.

For each node i , depending on whether it is being considered as source or destination node, the source and destination projections are computed as $W_s h_i^{(t,r)}, W_d h_i^{(t,r)} \in \mathbb{R}^{d_k}$, where $W_s, W_d \in \mathbb{R}^{D \times d_k}$ are trainable projection matrices for each head, so that the representation of node i at head r is given by $\tilde{h}_{i,s}^{(t,r)}, \tilde{h}_{i,d}^{(t,r)} \in \mathbb{R}^{d_k}, r = 1, \dots, R$.

Similarly, $e_{ij} \in \mathbb{R}^{d_e}$ denotes the attribute vector associated with edge (i, j) , where d_e is the edge feature dimension. These attributes are projected using $W_e^{(r)} e_{ij} \in \mathbb{R}^{d_k}$, where $W_e^{(r)} \in \mathbb{R}^{d_e \times d_k}$ is a trainable matrix for head r , with the projected edge features $\tilde{e}_{ij}^{(r)} \in \mathbb{R}^{d_k}$.

For each edge (i, j) and head r , the intermediate attention feature vector is formed:

$$\tilde{u}_{ij}^{(t,r)} = \text{LeakyReLU} \left(\tilde{h}_{i,s}^{(t,r)} + \tilde{h}_{j,d}^{(t,r)} + e_{ij}^{(r)} \right) \quad (11)$$

If no edge attributes are used, the term $e_{ij}^{(r)}$ is omitted.

The scalar attention score is then computed using a shared learnable projection vector:

$$a_{ij}^{(t,r)} = W_a^\top \tilde{u}_{ij}^{(t,r)} \quad (12)$$

where $W_a \in \mathbb{R}^{d_k}$ is shared across all heads. Although the projection vector is shared, the resulting attention scores remain head-specific because $\tilde{u}_{ij}^{(t,r)}$ differ across heads.

The normalized attention weight for node i receiving information from node j at head r is computed by softmax over the incoming neighborhood, where $\mathcal{N}(i)$ denotes the set of neighbors sending messages to node i :

$$\alpha_{ij}^{(t,r)} = \frac{\exp(a_{ij}^{(t,r)})}{\sum_{j' \in \mathcal{N}(i)} \exp(a_{ij'}^{(t,r)})} \quad (13)$$

The value features are also split by head. Let $v_j^{(t,r)} \in \mathbb{R}^{d_k}$ denote the value representation of node j at head r , obtained by reshaping the node features into R heads. The head-wise spatial context for node i is then

$$m_i^{(t,r)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(t,r)} v_j^{(t,r)} \quad (14)$$

Finally, the outputs of all heads are concatenated to form the multi-head spatial representation:

$$H_i^{\text{spatial},(t)} = \parallel_{r=1}^R m_i^{(t,r)} \in \mathbb{R}^D \quad (15)$$

D. Gated Multi-Hop Spatial Propagation

A key limitation of standard graph attention is its reliance on single-hop aggregation per layer, making the receptive field dependent on the number of stacked layers. To mitigate this, a gated multi-hop propagation mechanism is introduced that is inspired by GeniePath [15]. While GeniePath models depth implicitly through layer stacking, the proposed approach performs explicit multi-hop propagation within a single layer, effectively decoupling receptive field size from network depth.

At each time step t , let $H^{(t,0)} \in \mathbb{R}^{N \times D}$ denote the initial node representations obtained from the spatial attention module (the collection of $H_i^{\text{spatial},(t)}$ for all nodes), where N is the number of nodes and D is the feature dimension. A memory state $C^{(t,0)} = \mathbf{0} \in \mathbb{R}^{N \times D}$ is also initialized. It should be noted that this step is also performed in the context of multi-head attention, exactly as the spatial attention step described above. Therefore, to simplify the notation, the

process is described for all heads without tensor splitting to avoid excessive indexing.

The propagation proceeds for S steps. At step s , a one-hop message is computed using graph attention applied to the previous representation:

$$M_i^{(t,s)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(t,s)} H_j^{(t,s-1)} \quad (16)$$

where:

- $M_i^{(t,s)} \in \mathbb{R}^D$ is the aggregated message for node i ,
- $\alpha_{ij}^{(t,s)}$ are attention weights computed as described in the spatial attention module,
- $H_j^{(t,s-1)} \in \mathbb{R}^D$ is the representation of neighbor node j from the previous step.

To control the flow of information across propagation steps, an LSTM-style [19] gating mechanism is utilized. For each node i , a joint representation $Z_i^{(t,s)} = [H_i^{(t,s-1)} \parallel M_i^{(t,s)}]$ is formed, resulting in $Z_i^{(t,s)} \in \mathbb{R}^{2D}$.

The gating functions are defined as follows.

$$\begin{aligned} I_i^{(t,s)} &= \sigma(W_i Z_i^{(t,s)}), & F_i^{(t,s)} &= \sigma(W_f Z_i^{(t,s)}), \\ O_i^{(t,s)} &= \sigma(W_o Z_i^{(t,s)}), & G_i^{(t,s)} &= \tanh(W_g Z_i^{(t,s)}) \end{aligned} \quad (17)$$

where:

- $I_i^{(t,s)}, F_i^{(t,s)}, O_i^{(t,s)}, G_i^{(t,s)} \in \mathbb{R}^D$,
- $W_i, W_f, W_o, W_g \in \mathbb{R}^{2D \times D}$ are learnable weight matrices,
- $\sigma(\cdot)$ denotes the sigmoid activation function.

The memory and hidden states are updated as:

$$C_i^{(t,s)} = F_i^{(t,s)} \odot C_i^{(t,s-1)} + I_i^{(t,s)} \odot G_i^{(t,s)} \quad (18)$$

$$H_i^{(t,s)} = O_i^{(t,s)} \odot \tanh(C_i^{(t,s)}) \quad (19)$$

where $\odot$ denotes element-wise multiplication. During training, a dropout layer is applied to $H^{(t,s)}$ at every step s .

After S propagation steps, the final updated spatial representation is:

$$H^{\text{spatial},(t)} = H^{(t,S)} \in \mathbb{R}^{N \times D} \quad (20)$$

This representation is then used in the subsequent spatio-temporal fusion step. This formulation enables multi-hop aggregation while maintaining stability through gated updates, explicitly controlling how new neighborhood information is incorporated at each step.

E. Node-Adaptive Spatio-Temporal Fusion

The final node representation is obtained by combining temporal and spatial components using a node-specific adaptive mechanism:

$$H_i^{(t)} = \lambda_i \cdot H_i^{\text{temporal},(t)} + (1 - \lambda_i) \cdot H_i^{\text{spatial},(t)} \quad (21)$$

where:

- $H_i^{\text{temporal},(t)}, H_i^{\text{spatial},(t)} \in \mathbb{R}^D$,
- $\lambda_i \in (0, 1)$ is a learnable scalar parameter associated with node i .

This adaptive fusion allows each node to balance temporal dynamics and spatial interactions based on its local characteristics, which enables more flexible and expressive modeling. During training, a dropout layer is applied to H after this step.

F. Encoder Layer

The model consists of L stacked spatio-temporal encoder layers. Each layer refines the node representations by combining spatio-temporal attention, node-adaptive fusion, and a feedforward transformation.

Let $H^{(l-1)} \in \mathbb{R}^{T \times N \times D}$ denote the input to the l -th layer. After applying spatio-temporal attention and node-adaptive fusion, $\tilde{H}^{(l)} \in \mathbb{R}^{T \times N \times D}$ is obtained. A position-wise feedforward network is then applied:

$$\text{FFN}^{(l)}(x) = W_{p2}^{(l)}(\text{ReLU}(W_{p1}^{(l)}x + b_{p1}^{(l)})) + b_{p2}^{(l)} \quad (22)$$

where:

- $W_{p1}^{(l)} \in \mathbb{R}^{D \times D_f}$, $W_{p2}^{(l)} \in \mathbb{R}^{D_f \times D}$,
- $b_{p1}^{(l)} \in \mathbb{R}^{D_f}$, $b_{p2}^{(l)} \in \mathbb{R}^D$,
- D_f is the hidden dimension of the feedforward layer.

The output of the encoder layer is computed using a residual connection and normalization:

$$H^{(l)} = \text{LayerNorm}\left(\tilde{H}^{(l)} + \text{FFN}^{(l)}\left(\text{LayerNorm}(\tilde{H}^{(l)} + H^{(l-1)})\right)\right) \quad (23)$$

Stacking L such layers produces the final representation $H^{(L)} \in \mathbb{R}^{T \times N \times D}$. In each encoder layer, both $H^{(l-1)}$ and the output of $\text{FFN}^{(l)}$ are passed through a dropout layer during training.

G. Prediction Head

The prediction is performed using the representation at the final time step $H^{(L,T)} \in \mathbb{R}^{N \times D}$. A shared multi-layer perceptron (MLP) is applied independently to each node. The MLP consists of two fully connected layers with a nonlinear activation, where $W_{pred1} \in \mathbb{R}^{D \times \frac{D}{2}}$, $b_{pred1} \in \mathbb{R}^{\frac{D}{2}}$, $W_{pred2} \in \mathbb{R}^{\frac{D}{2} \times 1}$, and $b_{pred2} \in \mathbb{R}$. Let $\hat{y}_i$ be the prediction for node i , with the MLP collapsing the feature dimension to a single dimension where the node-level prediction lies.

$$\hat{y}_i = W_{pred2} \text{ReLU}\left(W_{pred1} H_i^{(L,T)} + b_{pred1}\right) + b_{pred2} \quad (24)$$

IV. RESULTS

A. Dataset Preparation & Graph Construction

CAP-STN is evaluated on the LargeST dataset [4], a large-scale benchmark for traffic forecasting constructed from loop detector measurements collected across the California highway network. The dataset contains 8,600 sensors (N) and spans five years at 5-minute intervals, providing both high spatial resolution and long-term temporal coverage, making it suitable for evaluating models under realistic large-scale spatio-temporal settings.

The data is represented as $\mathbf{X} \in \mathbb{R}^{T \times N \times F_{dyn}}$, where F_{dyn} is the number of dynamic features per node, which, for the LargeST dataset, is equal to 1, corresponding to traffic volume.

As described in Section III-A, the data at each time step are augmented with periodic time features. Each node is associated with geographic coordinates, $Coor \in \mathbb{R}^{N \times 2}$, representing longitude and latitude. These coordinates are used to construct 2D spatial encodings as described in Section III-B, capturing relative spatial relationships between nodes. These features, whose dimension is determined from model hyperparameters, are concatenated with the dynamic inputs at each node and time step, resulting in $\mathbf{X} \in \mathbb{R}^{T \times N \times F}$ where F includes F_{dyn} , periodic time features, and spatial encoding. In addition, standard sinusoidal positional encoding is applied along the temporal dimension to preserve temporal ordering.

The spatial directional graph is derived from a precomputed adjacency matrix provided with the dataset, which encodes pairwise relationships between sensors based on the underlying road network. The adjacency matrix is converted into a sparse edge set $\mathcal{E} = \{(i, j)\}$, where an edge exists if the corresponding entry in the adjacency matrix is non-zero. Each edge (i, j) is associated with a scalar weight extracted from the adjacency matrix $e_{ij} \in \mathbb{R}^{d_e}$, with $d_e = 1$, which reflects the strength of connectivity between nodes. These edge attributes are incorporated into the spatial attention mechanism.

Following standard practice, sliding windows are used to generate training samples. Each input sequence is defined as $\mathbf{X}_{seq} \in \mathbb{R}^{L_{in} \times N \times F}$, where L_{in} is the input sequence length. The model is trained to predict the traffic value at the next time step for all nodes.

B. Model Configuration

CAP-STN is implemented using stacked spatio-temporal encoder layers. The model processes input sequences $\mathbf{X}_{seq} \in \mathbb{R}^{L_{in} \times N \times F}$ and produces node-level predictions $\hat{\mathbf{y}} \in \mathbb{R}^N$. The main hyperparameters used in all experiments are as follows.

- Number of frequency bands: $k = 8$
- Number of harmonics: $d_h = 4$
- Hidden dimension: $D = 64$
- Number of attention heads: $R = 4$
- Hidden dimension of the feedforward layer: $D_f = 4 \times D$
- Number of encoder layers: $L = 2$
- Spatial propagation steps: $S = 3$
- Input sequence length: $L_{in} = 12$
- Dropout rate for training: 0.1

C. Experimental Setup

CAP-STN is evaluated against MTGNN and GMAN, using standard regression metrics MSE (Mean Squared Error), MAE (Mean Absolute Error), and RMSE (Root Mean Square Error). The data is temporally split into 80% training and 20% testing sets. All models are trained and evaluated under the same data split and preprocessing pipeline to ensure fair comparison.

D. Performance Evaluation

1) *Baseline Comparison:* As shown in Table I, CAP-STN consistently outperforms both MTGNN and GMAN across all evaluation metrics, demonstrating the effectiveness of combining continuous spatial encoding, periodic temporal features, enlarged receptive fields, and adaptive fusion mechanisms.

TABLE I
PERFORMANCE COMPARISON WITH BASELINE MODELS

Model	MSE	MAE	RMSE
MTGNN	0.017391	0.080145	0.131875
GMAN	0.018664	0.091726	0.136615
CAP-STN	0.016631	0.077791	0.128963

TABLE II
ABLATION STUDY OF MODEL COMPONENTS

Model Variant	MSE	MAE	RMSE
w/o periodic temporal encoding	0.016995	0.079716	0.130364
w/o 2D spatial encoding	0.017154	0.079426	0.130975
w/o gated multi-hop spatial propagation	0.017790	0.081103	0.133379
w/o node-adaptive fusion	0.016807	0.078464	0.129642
full model	0.016631	0.077791	0.128963

2) *Ablation Study*: To assess the contribution of each component, an ablation study is conducted by systematically removing key modules: (1) periodic temporal encoding, (2) 2D spatial encoding, (3) gated multi-hop spatial propagation, (4) node-adaptive fusion. As depicted in Table II, which summarizes the results of the ablation studies, removing each component leads to a degradation in performance. This confirms that all proposed modules contribute meaningfully to the final model.

V. CONCLUSION

This work presents CAP-STN, a spatio-temporal forecasting framework that addresses key limitations of existing graph-based and attention-based models. The proposed approach integrates periodic temporal encoding, continuous 2D spatial encoding, gated multi-hop spatial propagation, and node-adaptive fusion within a unified architecture.

The introduction of continuous spatial encoding enables the model to capture geographic relationships beyond discrete graph connectivity, while periodic temporal features provide explicit modeling of recurring patterns. The gated multi-hop spatial propagation mechanism allows controlled multi-hop aggregation without requiring deep stacking, improving stability. In addition, the node-adaptive fusion mechanism enables dynamic balancing of spatial and temporal contributions based on node-specific characteristics.

Experimental results on the LargeST dataset demonstrate that CAP-STN consistently outperforms MTGNN and GMAN in multiple evaluation metrics. Ablation studies further confirm the contribution of each component to overall performance.

Future work will explore extending the proposed framework to additional domains, incorporating richer edge and node attributes, and improving scalability for even larger graph structures. Another promising direction is to integrate adaptive graph construction and learning mechanisms to further enhance the spatial representation of dynamic graph data.

REFERENCES

[1] S. Pouyanfar, S. Sadiq, Y. Yan, H. Tian, Y. Tao, M. P. Reyes, M.-L. Shyu, S.-C. Chen, and S. Iyengar, "A survey on deep learning: Algorithms,

techniques, and applications," *ACM Computing Surveys*, vol. 51, no. 5, 2018.

[2] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 753–763.

[3] C. Zheng, X. Fan, C. Wang, and J. Qi, "Gman: A graph multi-attention network for traffic prediction," *AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, pp. 1234–1241, Apr. 2020.

[4] X. Liu, Y. Xia, Y. Liang, J. Hu, Y. Wang, L. BAI, C. Huang, Z. Liu, B. Hooi, and R. Zimmermann, "Largest: A benchmark dataset for large-scale traffic forecasting," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 75 354–75 371.

[5] S.-C. Chen, M.-L. Shyu, and C. Zhang, "An intelligent framework for spatio-temporal vehicle tracking," in *IEEE International Conference on Intelligent Transportation Systems*, 2001, pp. 213–218.

[6] S.-C. Chen, M.-L. Shyu, S. Peeta, and C. Zhang, "Learning-based spatio-temporal vehicle tracking and indexing for transportation multimedia database systems," *IEEE Transactions on Intelligent Transportation Systems*, vol. 4, no. 3, pp. 154–167, 2003.

[7] X. Li, S.-C. Chen, M.-L. Shyu, and B. Furht, "Image retrieval by color, texture, and spatial information," in *Proceedings of the 8th international conference on distributed multimedia systems*, 2002, pp. 152–159.

[8] S.-C. Chen, S. Sista, M.-L. Shyu, and R. Kashyap, "Augmented transition networks as video browsing models for multimedia databases and multimedia information systems," in *11th International Conference on Tools with Artificial Intelligence*, 1999, pp. 175–182.

[9] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting," in *27th International Joint Conference on Artificial Intelligence*, ser. IJCAI'18. AAAI Press, 2018, p. 3634–3640.

[10] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, "T-gcn: A temporal graph convolutional network for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.

[11] J. Bai, J. Zhu, Y. Song, L. Zhao, Z. Hou, R. Du, and H. Li, "A3t-gcn: Attention temporal graph convolutional network for traffic forecasting," *ISPRS International Journal of Geo-Information*, vol. 10, no. 7, 2021.

[12] A. Roy, K. K. Roy, A. Ahsan Ali, M. A. Amin, and A. M. Rahman, "Sst-gnn: simplified spatio-temporal traffic forecasting model using graph neural network," in *Pacific-asia conference on knowledge discovery and data mining*. Springer, 2021, pp. 90–102.

[13] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," in *International Conference on Learning Representations*, 2020.

[14] —, "Self-attention with functional time representation learning," *Advances in neural information processing systems*, vol. 32, 2019.

[15] Z. Liu, C. Chen, L. Li, J. Zhou, X. Li, L. Song, and Y. Qi, "Geniepath: graph neural networks with adaptive receptive paths," in *Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI'19/IAAI'19/EAAI'19. AAAI Press, 2019.

[16] X. Huang, J. Wang, Y. Lan, C. Jiang, and X. Yuan, "Md-gcn: A multi-scale temporal dual graph convolution network for traffic flow prediction," *Sensors*, vol. 23, no. 2, 2023.

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.

[18] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations*, 2022.

[19] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.